11. Generics: Type Parameters and Constraints

11. Generics: Type Parameters and Constraints

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `11-generics`

Learning objectives

- Write generic functions with type parameters
- Use constraints from constraints package / comparable
- Know when generics help vs when interfaces suffice
- Avoid over-generic APIs

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Generics remove `any` casts for containers and algorithms while keeping type safety. Used well, they clarify APIs; used poorly, they obscure them.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Type parameters appear in `[T constraints]` on funcs and types.
- `comparable` enables == and map keys.
- Approximation elements `~int` include defined types based on int.
- Prefer interfaces when behavior is method-based; generics for structure.
- Inference often fills type args — specify when the compiler cannot.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-generics: && cd /tmp/gw-generics:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Map[A, B any](xs []A, f func(A) B) []B {
    out := make([]B, len(xs))
    for i, v := range xs {
        out[i] = f(v)
    }
    return out
}

func main() {
    fmt.Println(Map([]int{1, 2, 3}, func(x int) int { return x * 2 }))
}
```

Run it:

```
go run .
```

Expected output:

```
[2 4 6]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write generic Keys[K comparable, V any].
2. Write Sum with a Number constraint union.
3. Replace an `any` + type assert helper with a generic.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Type parameters

```
func Keys[K comparable, V any](m map[K]V) []K {
    out := make([]K, 0, len(m))
    for k := range m {
        out = append(out, k)
    }
    return out
}
```

Constraints

```
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](xs []T) T {
```

```
    var s T
    for _, v := range xs {
        s += v
    }
    return s
}
```

`comparable` is required for map keys and `==`.

When not to use generics

If a small interface expresses the behavior better, prefer interfaces. Generics shine for containers, algorithms, and removing `any` casts.

Failure modes to watch while coding

- Genericizing every function “just in case”
- Forgetting `comparable` for map keys
- Fighting the type checker instead of simplifying the API

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python typing generics are optional; Go generics are enforced.
- No *args magic — constraints declare what operations exist.
- Don’t genericize every function; concrete types stay readable.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

The standard library uses generics in slices, maps helpers — read those packages as style guides.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Generics: Type Parameters and Constraints” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Genericizing every function “just in case”
- Forgetting `comparable` for map keys
- Fighting the type checker instead of simplifying the API

Exercises

Work in `exercises/11-generics/`. Read that folder’s `README.md` before you code.

What you will build

Implement a generic Map (or Keys) as specified by the exercise tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/11-generics/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does `comparable` allow?
2. When prefer interface methods over type parameters?
3. What does `~int` mean in a constraint?

Further reading

- https://go.dev/doc/tutorial/generics
- https://go.dev/blog/intro-generics